

PB158 — Practicum in Operating Systems: course orientation

Contents

1 Who teaches this course

2 Why this course exists

3 What the course covers

4 How you are assessed

4.1 The two components

4.2 The three assignments

4.3 The loop

4.4 What passing buys you

4.5 Who grades you, and how that fails

4.6 What this course is betting

5 What must work on your machine

5.1 Windows

5.2 Linux

5.3 Verifying the install

6 Where to keep learning

7 Open questions

8 References

9 Appendix — sources from the author's working notes

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

Reading this document: *[S<n>] cites the numbered source list at the end. [inference] marks a claim that follows from those sources but is not stated by them. [speculative] marks the author's judgement, offered as judgement.*

This handout is the durable version of lecture 1, delivered on 15 September 2026 at the Faculty of Informatics, Masaryk University. It carries everything from the slides. It also carries the material that did not fit. That is the full reading list with URLs, the complete install instructions for both platforms, and the questions the lecture leaves open. [speculative]

1 Who teaches this course

Ondřej Krajíček (krajicek@mail.muni.cz) and a co-lecturer teach PB158. Three teaching assistants join us in the labs. IS MUNI carries the full roster with contact details once the course opens; I keep it there rather than in a deck that is published outside the university. Write to either lecturer directly. You do not need a reason and you do not need to wait for a seminar.

Two systems, two directions, and the contrast is worth memorising on day one. Course material — slides, this handout, the assignment briefs — comes **from IS MUNI**, and that is the single place to look. Your work goes **to FI MUNI GitLab**, at gitlab.fi.muni.cz. The deck is also published to a public site as a by-product. IS MUNI is the student-facing route, and the one that will be kept current. [speculative]

There are fewer than ten of you this semester. I say so because the cohort size decides what the course can do with your mistakes. With fewer than ten of you, three weeks of the term are review lectures. Each one is built from the specific defects in your submissions. The lecturers read those submissions in the three days between the deadline and the review. Two hundred submissions cannot be read that way in three days. [speculative]

2 Why this course exists

You can write Python. You can write C. You have had operating systems as theory, so you can define a system call, a process and virtual memory. None of you has attached a debugger or a tracer to a running program. That combination — able to define the thing, never having watched it happen — is the gap this course is built around. [speculative]

It is not a gap in you, and it has a date. Andrew Tanenbaum's retrospective on MINIX records the sequence. John Lions wrote a line-by-line commentary on the UNIX version 6 kernel in 1977. It became the textbook of hundreds of universities. The 1979 AT&T release then shipped a licence forbidding anyone to write about the source or teach it. Operating systems courses, in his words, went back to theory-only mode or moved to toy simulators [S5]. DTrace patches the system only when a probe is enabled. That property is "what made DTrace safe to ship enabled on production systems" [V23]. This course puts a tracer of that kind in your hands. You attach `strace` on Linux, or NtTrace on Windows, to a running program. You read what it printed. You do this in the same term that you define a system call. [speculative]

The thesis has a precise technical statement, and the two words already exist. *Monitoring* "answers questions you already knew to ask" [V2] — somebody chose the metric, set the threshold, wrote the alert. A written theory test is monitoring. *Observability* is "the ability to investigate a system by asking any question about its behavior without needing to predict those questions in advance" [V1]. The word is older than its current marketing. It "was introduced by the Hungarian-American engineer **Rudolf E. Kálmán** in 1960 for linear dynamic systems" [V3]. That is worth knowing when you meet the word in a job advertisement. [speculative]

The claim that you can recite more than you can predict is not rhetoric. Rozenblit and Keil named and measured it as the illusion of explanatory depth. People rate their own understanding of a thing lower after they attempt to explain it than before. The effect is strongest for explanatory knowledge and weakest for plain facts, which is the exact shape of a theory-only education [S1]. In the lecture I ask you to do this silently, for thirty seconds. Name the steps between a Python `f.write()` returning and the bytes reaching the disk. Generating the explanation is what reveals the gap. Being told about the gap does not. [inference]

A practicum differs from a lecture course in what it asks you to produce. A lecture course asks you to reproduce somebody else's explanation. A practicum asks you to produce an observation and defend it. Crouch, Fagen, Callan and Mazur measured the

difference with classroom demonstrations in physics. Passively watching a demonstration produced very little learning. Predicting the outcome first produced a measurable gain [S2]. Predict, then look. [inference]

I assert that thesis in this lecture and prove nothing. The proof is the 65-minute lecture that follows immediately, which opens with three lines of Python whose output disappears when it is redirected. [speculative]

3 What the course covers

One idea carries all eight topic areas. Every operating-system interface keeps its promise only under conditions nobody wrote down. The written half of the contract says what a call does. The unwritten half — the scope clause — says when it does that. "An abstraction over the operating system is a **contract with a silent scope clause**. The contract says what the call does. The scope clause — the part nobody writes down — says under which conditions it does that. The leak is not the abstraction failing; it is the program leaving the scope." [V4] The method follows from that. You cannot reason your way to the guilty layer. You record each boundary crossing and count it. "You cannot deduce which layer is lying; you have to make the boundary crossings visible and count them." [V5] When the manual and the trace disagree, the trace wins. "A library's documentation is a claim; the syscall it issues is a fact." [V21] [inference]

Topic 1 — I/O: files and the network. One file write and one network request, followed down through every layer between your code and the device. The general rule: "Almost every fast OS interface returns when the kernel has *taken responsibility* for the work, not when the work has happened." [V6] Sockets sit here because `read` and `write` operate on both, and the platforms diverge sharply: "In Winsock applications, a socket descriptor is not a file descriptor and must be used with the Winsock functions." [V7] The course starts here for two reasons, and both are mine. Everyone has written a file, so the topic needs no motivating. And following one `write()` down is what forces the system-call boundary into view and makes a tracer necessary rather than decorative. Teach `strace` in week one and a student learns a command with no reason to want it. [speculative]

Topic 2 — The system-call boundary. Where your program stops and the operating system begins. Linux publishes the system-call interface itself as the stable contract. Windows publishes Win32 and reserves the layer beneath it. The correction that stops this reading as "Linux is right": "Both are stable at the layer they nominate. The disagreement is about *which layer is nominated*" [V8]. [inference]

Topic 3 — Observing a running system. The tools properly: `strace` on Linux, NtTrace on Windows, and how to read what they print. I teach the limit with the tool, because "A tracer is not a window onto a running program. It is an edit to that program." [V22] A tracer adds code, so it changes the timing it reports. [inference] And a tracer at the system-call boundary sees only the work that crosses it: "A syscall-boundary tracer cannot see work that does not cross the syscall boundary — which is why `io_uring`, whose submissions live in shared memory rings, is invisible to `strace`" [V25]. This design is published rather than invented here. A published graduate course in operating systems "teaches kernel behaviour through DTrace and hardware performance counters rather than kernel hacking" [V9]. Its own designers described it. They recorded its equipment failures too [S3]. [inference]

Topic 4 — Virtual memory and the page cache. Where the bytes you wrote actually went, and why memory your program freed is still held. "A mapping can exist with no physical page behind it. The page is materialized on first touch, by a fault." [V17] Allocation gives you address space, not memory, and a page fault is the normal mechanism rather than an error. The practical skill is reading a memory figure and saying which question it answers. [speculative]

Topic 5 — Memory protection, shared memory and IPC. "Every load, store, and instruction fetch a process performs is checked before it completes." [V18] The memory management unit checks this in hardware, against permission bits the kernel wrote. The compiler, the language runtime and the kernel code perform no part of the check. The kernel writes the policy and the hardware enforces it. That is why isolation is cheap enough to be unconditional, and why a segmentation fault is the protection mechanism working rather than failing. [inference] The second half of the topic covers the deliberate crossing: shared mappings, pipes, message queues and signals. [speculative]

Topic 6 — Processes, threads and scheduling. What the operating system actually schedules, and how language runtimes redefine the thread you think you are using. "A userspace-scheduled unit of concurrency produces no system call when it is created and no system call when it yields, so a syscall tracer shows nothing at all where the student expects to see concurrency." [V19] The absence is the finding [inference]. Reading an absence as evidence is a genuinely hard skill to learn, and this topic is where the course teaches it. [speculative]

Topic 7 — Post-mortem analysis, with and without an agent. "Tracing shows you a running system. A dump is all you get once it is already dead." [V20] A dump is a photograph rather than a recording, and you reason backward from it. The agent half is deliberately tool-specific rather than a blanket claim. An agent adds breadth over a large

body of evidence, and it adds little on a single stack frame [V26]. "A library's documentation is a claim; the syscall it issues is a fact" applies unchanged [V21]. The agent reads only what the trace and the dump captured [V27].

Topic 8 — Leaks in the abstraction. The synthesis lecture at the end of term. "The leak is not the abstraction failing; it is the program leaving the scope." [V4] What you leave the course with is a personal list of scope clauses you have watched a program step outside of. [inference]

There is no fixed week for each topic. Three weeks of the term are review lectures built from what your submissions actually got wrong, so their content is not knowable in September. You get the topics and the order. The pace follows the cohort. [speculative]

4 How you are assessed

4.1 The two components

PB158 has two components, and you must pass both. The **course credit** (*zápočet* in IS MUNI) is awarded pass or fail. The university's regulations record it with the wording "requirements fulfilled" or "requirements not fulfilled" [S9]. The **exam** (*zkouška*) is the graded component at the end of the term. IS MUNI and the faculty's paperwork use these Czech names. Look for those words, not for an English translation, when you open your own record. [speculative]

In PB158 you earn the course credit by submitting at least one of the three homework assignments. You do not have to pass one. The checker can fail your submission, and it still earns the course credit. This is the course's own rule, not a university regulation. [speculative]

You write the exam in a classroom under controlled conditions. The rule about AI is simple: you work unaided. It covers models you bring yourself, and it covers agent harnesses. [speculative]

4.2 The three assignments

Three assignments per semester, in teams of two to three, submitted to FI MUNI GitLab at `gitlab.fi.muni.cz`. Each is pass or fail. There are no points. [speculative]

ASSIGNMENT	SET	DUE, MIDNIGHT	REVIEWED
1	Tue 22 Sep 2026	Fri 2 Oct 2026	Tue 6 Oct 2026
2	Tue 6 Oct 2026	Fri 16 Oct 2026	Tue 20 Oct 2026
3	Tue 20 Oct 2026	Fri 30 Oct 2026	Tue 3 Nov 2026

Read the dates rather than derive them [speculative]. Set-to-due is **ten days**, not a fortnight — planning against "two weeks" costs you a weekend. The deadline sits three days before the review lecture on purpose, so the lecturers can read the submissions and build the review from them. A deadline landing on the lecture itself would deliver the data at the moment it was needed rather than in time to use it. [speculative]

Nothing is set in week 1. The cadence begins on 22 September. [speculative]

4.3 The loop

You may resubmit as many times as you like until the deadline. A fail before the deadline costs you time and nothing else. That makes the homework a loop rather than a test with retries: submit, be told what is wrong, fix it, submit again. [speculative]

One rule inside the loop is load-bearing. The checker returns **defects, not remedies**. It will say that your program loses its output when the output is redirected. It will not say to change line 42. [speculative]

The standard formulation of this feedback pattern requires the critic to "identify specific deficiencies and suggest concrete revisions, not just approve/reject" [V15]. This course keeps the first half and drops the second deliberately. A checker that hands you the fix has done the assignment for you. The resulting pass measures nothing. [speculative]

A consequence follows that decides how much you get out of the course. The review lectures are built from what the cohort got wrong. But a team that iterates until the checker goes green has, by the deadline, removed every defect the checker tests for. The passing commit is by construction the one with nothing left that the checker can catch. Build the review from final submissions and it shows a cohort that did everything right. The teaching artifact is therefore the **sequence of attempts**, which your Git history already records. Push your failures. This is the course's own thesis applied to the course: look at what the system recorded, not at the state it ended in. [speculative]

4.4 What passing buys you

The exam has two parts: an irreducible core that every student sits unaided, and a removable part. **Passing an assignment removes that assignment's subject matter from your exam.** The course fixes which subject each assignment removes, and you do not choose it. On the day I set an assignment, you know exactly what passing it buys. It is a smaller syllabus, not merely a shorter paper. Pass all three and only the core remains. [speculative]

The submission is the team's, and the reduction applies to every member of the team. Your exam therefore depends on your teammates, which is worth knowing on day one rather than in January. The cost is real and I accepted it knowingly. A member who contributed nothing skips exam material on precisely the topic they did not learn. [speculative]

The core is what bounds that, and it is load-bearing for a second reason. You do the homework where AI is available. The exam is the only place this course measures you without it. [speculative]

"When a measure becomes a target, it ceases to be a good measure" [V16]. An unlimited-retry checker is a target by construction, so a determined team can converge on satisfying it without learning the subject. [inference] Iterating against a verifier is itself part of the subject: you read each failure and converge on a specification you did not write. This course teaches that deliberately. [speculative] The unaided core is the only thing that distinguishes it from making a checker happy. [speculative]

4.5 Who grades you, and how that fails

An AI grades both the exam and the homework. A human stays on the loop. That phrase is weaker than it sounds and you should know exactly how. "The defining feature is negative: the action does *not* wait for the human. It proceeds by default, and the human's role is to catch it — not to authorise it." [V10] Nobody checks before a fail is a fail. The prevailing industry phrase, *human in the loop*, describes the arrangement where the action does wait for a person. Using it here would overstate the oversight. [inference]

I name the asymmetry in the lecture rather than leave you to discover it. You may not use AI in the exam, and an AI grades you. The defensible version is also the true one. The exam measures what *you* can do unaided. Grading is a different task — assessing finished work against a rubric — and it runs with a human attached. [speculative]

The evidence on AI grading does not support a stronger claim than that. Teachers working with automated scoring "distrusted automated scoring and emphasized the need for human oversight" [S4]. This course grades pass or fail and returns written defects, which is my decision rather than a finding. [speculative] I avoid a numeric score for a reason of my own: I could not defend a one-point difference to a student who challenged it. [speculative]

Three named failure modes, and this is the first system the course asks you to examine rather than trust:

- **Vigilance decays.** Mackworth's observers watched "a pointer stepping once per second" for two hours, and had to catch its rare double jumps [V24]. In that task, "detection fell from about 85% at the outset to about 75% within the first 30 minutes, and continued to decline across the remainder of the two-hour vigil" [V11]. Bainbridge generalised the problem in 1983. The better the automation, the harder the human monitor's job becomes. The paper remains a live description of automation-related accidents [S8]. [inference]
- **Review becomes ritual.** "gate everything and you train the reviewer to click through, which is a rubber stamp with an audit trail" [V12]. The course plans to review passes rather than attempts. A pass is what buys exam relief, and a failed attempt costs a student a retry and nothing else. But that queue is by construction full of work that already succeeded, and that is a poor input to a human monitor. [inference]
- **A critic invents faults on good work.** "For tasks where models started strong ($\geq 75\%$ accuracy), the critique loop was devastating for both models. The critic, primed to find errors, invented them." [V14] This is the failure that bites a strong submission specifically. [inference]

The structural repair is not more vigilance. "the human's job is to adjudicate a raised alarm, not to detect the anomaly" [V13] — so tell us when a result is wrong. An arrangement that reviews passes never shows a human a wrongly failed team. Raising it yourself is the only thing that puts it in front of one. [inference]

4.6 What this course is betting

The published evidence does not agree with the design, and the lecture says so rather than waiting to be asked.

- A team resubmits a one-line edit every few minutes to see which test flips [speculative]. Lawrence, Foss and Urazova found that submission limits and regression penalties reduce that behaviour by up to 85%. Students themselves prefer submission limits [S6]. [inference]

- Luxton-Reilly, Tempero and Arachchilage name the behaviour. Rapid feedback leads some students to focus on passing the test cases through fast repeated submissions. They do not systematically improve the program [S18]. [inference]
- Kirschner, Sweller and Clark compared minimal guidance with strong guidance. Under minimal guidance a learner with low prior knowledge learns less of the material than under strong guidance. Their own wording is "minimally guided instruction is less effective and less efficient than instructional approaches that place a strong emphasis on guidance of the student learning process" [S22]. The disadvantage recedes only when learners hold enough prior knowledge to guide themselves internally [S22]. [inference] A theory-only cohort has low prior knowledge. Reading this room that way is my judgement rather than their finding. [speculative]
- Teaching to mastery with retries is not new either. The Keller Plan outperformed conventional lecturing on outcome measures for decades, and its grading labour is what ended it [S7]. [inference]
- Unlimited resubmission is not new and its earlier version shipped with a counterweight. Edwards' Web-CAT allowed unlimited submissions up to the deadline. It assessed a student's program only by the tests the student had written. The route to knowing what was wrong therefore ran through writing a test [S17]. [inference]

PB158 takes the unrestricted arm anyway, and substitutes its own counterweight: feedback phrased as defects rather than remedies, and an unaided exam core. Those two are therefore not good practice. They are the load-bearing parts of the design. If the bet is wrong, it will be wrong here. [speculative]

5 What must work on your machine

The Windows side is **Microsoft native by decision**: no MinGW and no MSYS2. The reason is the crash-dump topic later in the term. WinDbg wants MSVC's PDB symbols. A second toolchain emitting a different symbol format would cost students exactly where they first meet a debugger. [speculative]

5.1 Windows

TOOL	WHAT FOR	WHERE
Visual Studio Community	<code>cl.exe</code> , Windows SDK, PDB symbols. Tick Desktop development with C++	visualstudio.microsoft.com/vs/community
VS Code + C/C++ extension	the editor, on both platforms	code.visualstudio.com
Sysinternals Suite	Process Monitor, Process Explorer	learn.microsoft.com/sysinternals
NtTrace	native API tracing, the strace counterpart	github.com/rogerorr/NtTrace
WinDbg	crash dumps. Also ships in the Windows SDK	learn.microsoft.com/windows-hardware/drivers/debugger
WSL2	how you reach the Linux side from Windows	learn.microsoft.com/windows/wsl/install
Python	the language for the first assignments	python.org/downloads
uv	"An extremely fast Python package and project manager, written in Rust" [S21]	docs.astral.sh/uv

Visual Studio **Build Tools is not the right package** — it requires a Visual Studio licence, whereas Community is free for classroom use. All of these links were checked on 13 September 2026 and are re-checked before delivery. [speculative]

Two things cost an evening every year:

- **Visual Studio Community is large.** Microsoft's own figure is a minimum of 2.5 GB, and up to 210 GB depending on features. A typical installation needs 20–50 GB of free space [S19]. Check your free space first. Install Visual Studio Community before the first lab rather than tonight. The first assignment does not need it. [speculative]
- **`cl.exe` needs its own shell.** "Even a simple Visual Studio installation can set twenty or more environment variables. This complexity is why we strongly recommend that you use a developer command prompt shortcut" [S20]. In an ordinary PowerShell the error reads "not recognised", which looks like a failed install rather than a missing environment. [inference]

5.2 Linux

TOOL	WHERE
VS Code	<code>code.visualstudio.com</code>
gcc, make, libc headers	your distribution's build-essential equivalent
<code>strace</code> , <code>gdb</code>	your distribution's own packages
Python, uv	<code>docs.astral.sh/uv</code>

The course **dev container** carries the whole Linux side preinstalled. A dev container is "a running container with a well-defined tool/runtime stack and its prerequisites" [S10], described by a file committed with the code so that every student gets the same environment. On Windows it is also how you reach Linux, through WSL2. [inference]

One container-specific trap is worth knowing before it costs you an afternoon. A `ptrace`-based debugger or tracer inside a container needs the `SYS_PTRACE` capability and a relaxed seccomp profile. The VS Code dev-container documentation shows how to grant both [S11]. Julia Evans wrote the clearest diagnosis of the same failure, and it is on the reading list for that reason [S12]. [inference]

5.3 Verifying the install

On Linux, `gcc --version`, `strace --version`, `gdb --version` and `uv --version` should each print a version rather than an error. Run `strace -f -e trace=write python3 -c "print('hi')"`. Confirm that you see a `write` line. [speculative]

On Windows, open the **Developer Command Prompt for VS** from the Start menu. Do not use an ordinary PowerShell. Run `cl`. Confirm that it prints the compiler banner. Run `wsl --status` to confirm WSL2 is installed. Faculty machines are available for both platforms if your own hardware refuses — tell us rather than spending a week on it. [speculative]

6 Where to keep learning

The lecture names four. The full list is here, paired deliberately so that the Windows side is not an afterthought. All were active as of 13 September 2026. [speculative]

	SOURCE	WHAT IT IS
Linux, blog	Julia Evans , jvns.ca	Debugging, <code>strace</code> and the kernel from the outside in. Her piece on why <code>strace</code> does not work in Docker is the exact failure you will meet in the first lab [S12]
Windows, blog	Raymond Chen , <i>The Old New Thing</i>	Decades of first-party history on why Windows is the way it is, still publishing [S14]
Performance	Brendan Gregg	Flame graphs, eBPF, <code>Systems Performance</code> . This course's tracing-overhead figures come from his measurements [S13]
News	LWN.net	The weekly Linux kernel record, written by subject specialists. Free after two weeks [S15]
Podcast	Oxide and Friends	Bryan Cantrill and Adam Leventhal. Cantrill co-created DTrace, one of the two tracing traditions this course rests on [S16]

The reading list is not a list of things the lecturer happens to like. Two of these are sources the course's own material already rests on: the tracing-overhead figures are Gregg's [S13], and the container tracing diagnosis is Evans' [S12]. [inference]

7 Open questions

These are genuinely undecided or unstated as of 15 September 2026. Ask about them rather than assuming an answer. [speculative]

- **What is in the irreducible exam core.** It is what must be verified unaided, and it need not cover the whole subject — but its contents are not yet published. [speculative]

- **How much AI assistance the homework permits.** The rule stated is about the exam. The homework rule is a separate decision and is not yet made. Do not infer one from the other. [speculative]
- **Resits.** The university's regulations, not the course, govern how many attempts you have at the exam [S9]. [speculative]
- **Anonymity in the review lectures.** With two to four teams, a mistake discussed in a review is attributable on sight. The design still asks you to submit attempts you are unsure about. Raise it if it worries you. [speculative]

8 References

- **[S1]** Rozenblit, L. & Keil, F. (2002). *The misunderstood limits of folk science: an illusion of explanatory depth*. Cognitive Science 26, 521–562. <https://cogdevlab.yale.edu/sites/default/files/files/rozenblit%20&%20keil%20%202002.pdf>
- **[S2]** Crouch, C., Fagen, A., Callan, J. P. & Mazur, E. (2004). *Classroom demonstrations: learning tools or entertainment?* American Journal of Physics 72, 835. <https://works.swarthmore.edu/fac-physics/203/>
- **[S3]** Neville-Neil, G. V. et al. (2017). *Teaching Operating Systems with FreeBSD through Tracing, Analysis, and Experimentation*. USENIX ;login:. https://www.usenix.org/system/files/login/articles/login_spring17_03_neville-neil.pdf
- **[S4]** Tian et al. (2025). arXiv:2506.07955 — on teachers working with large-language-model grading in classrooms. Recorded title contradicted on re-reading, and no exact title confirmed in this build. <https://arxiv.org/abs/2506.07955>
- **[S5]** Tanenbaum, A. S. (2016). *Lessons Learned from 30 Years of MINIX*. Communications of the ACM. <https://www.cs.vu.nl/~ast/Publications/Papers/cacm-2016.pdf>
- **[S6]** Lawrence, Foss & Urazova (2023). On submission limits and regression penalties in automated assessment. ACM Transactions on Computing Education. <https://dl.acm.org/doi/10.1145/3591210>
- **[S7]** Eyre, H. L. (2007). *The Keller Plan and the Personalized System of Instruction*. The Behavior Analyst Today 8(3). <https://files.eric.ed.gov/fulltext/EJ800986.pdf>
- **[S8]** Bainbridge, L. (1983). *Ironies of Automation*. Automatica 19(6). <https://gwern.net/doc/sociology/technology/1983-bainbridge.pdf>
- **[S9]** Masaryk University. *MU Study and Examination Regulations*, current edition as published on the official notice board. The page carries the edition in force together with its later modifications. <https://www.muni.cz/en/about-us/official-notice-board/mu-study-and-examination-regulations>
- **[S10]** *Development Containers*. containers.dev. <https://containers.dev/>

- **[S11]** *Create a Dev Container*. Visual Studio Code documentation. <https://code.visualstudio.com/docs/devcontainers/create-dev-container>
- **[S12]** Evans, J. (2020). *Why strace doesn't work in Docker*. jvns.ca. <https://jvns.ca/blog/2020/04/29/why-strace-doesnt-work-in-docker/>
- **[S13]** Gregg, B. *brendangregg.com*. <https://www.brendangregg.com/>
- **[S14]** Chen, R. *The Old New Thing*. Microsoft Developer Blogs. <https://devblogs.microsoft.com/oldnewthing/>
- **[S15]** LWN.net. <https://lwn.net/>
- **[S16]** Cantrill, B. & Leventhal, A. *Oxide and Friends*. <https://oxidecomputer/podcasts/oxide-and-friends>
- **[S17]** Edwards, S. H. (Virginia Tech). Web-CAT project paper, `pos27-Edwards.pdf`. Exact title unconfirmed in this build. <https://web-cat.org/publications/pos27-Edwards.pdf>
- **[S18]** Luxton-Reilly, A., Tempero, E. & Arachchilage, N. ACE '23, Australasian Computing Education Conference. Exact title unconfirmed in this build. <https://valerio-terragni.github.io/assets/pdf/luxton-ace-2023.pdf>
- **[S19]** Microsoft Learn. *Visual Studio 2026 system requirements*. <https://learn.microsoft.com/en-us/visualstudio/releases/2026/vs-system-requirements>
- **[S20]** Microsoft Learn. *Use the MSVC toolset from the command line*. <https://learn.microsoft.com/en-us/cpp/build/building-on-the-command-line>
- **[S21]** Astral. *uv — an extremely fast Python package and project manager*. <https://docs.astral.sh/uv/>
- **[S22]** Kirschner, P., Sweller, J. & Clark, R. (2006). *Why Minimal Guidance During Instruction Does Not Work*. Educational Psychologist 41(2). https://itgs.ict.usc.edu/papers/Constructivism_KirschnerEtAl_EP_06.pdf
- **[V1], [V2], [V3]** *Observability vs Monitoring* — author's working notes.
- **[V4], [V5], [V6]** *Where the Abstraction Leaks* — author's working notes.
- **[V7]** *Sockets and Network I/O* — author's working notes.
- **[V8]** *Windows and Linux place the stable syscall boundary at different layers* — author's working notes.
- **[V9], [V23]** *Observing a Running System* — author's working notes.
- **[V10], [V11], [V12], [V13], [V24]** *Human on the Loop* — author's working notes.
- **[V14], [V15]** *Generative Actor Critic as an Agentic Orchestration Pattern* — author's working notes.
- **[V16]** *Goodhart's Law* — author's working notes.
- **[V17]** *Virtual Memory and the Page Cache* — author's working notes.

- **[V18]** *Memory Protection* — author's working notes.
- **[V19]** *Processes, Threads and Scheduling* — author's working notes.
- **[V20]** *Crash Dump Analysis on Windows and Linux* — author's working notes.
- **[V21], [V26], [V27]** *AI Agents as Debugging Operators* — author's working notes.
- **[V22], [V25]** *Every Tracing Mechanism Trades Observation Cost for a Blind Spot* — author's working notes.

9 Appendix — sources from the author's working notes

The talk cites material from a private research vault. Each entry below summarises one such note and lists every external source it draws on.

- `appendix/V1.md`
- `appendix/V2.md`
- `appendix/V3.md`
- `appendix/V4.md`
- `appendix/V5.md`
- `appendix/V6.md`
- `appendix/V7.md`
- `appendix/V8.md`
- `appendix/V9.md`
- `appendix/V10.md`
- `appendix/V11.md`
- `appendix/V12.md`
- `appendix/V13.md`
- `appendix/V14.md`
- `appendix/V15.md`
- `appendix/V16.md`
- `appendix/V17.md`
- `appendix/V18.md`
- `appendix/V19.md`
- `appendix/V20.md`
- `appendix/V21.md`
- `appendix/V22.md`
- `appendix/V23.md`

- [appendix/V24.md](#)
- [appendix/V25.md](#)
- [appendix/V26.md](#)
- [appendix/V27.md](#)